BookSwap: Funktionen und Entscheidungen

Finalisierungsphase

BookSwap bildet eine private Buchleihe in SQLite ab. Eine Buchausgabe enthält Titel, Sprache, Verlag, Autoren und Genres. Ein Exemplar steht für ein bestimmtes Buch im Besitz eines Mitglieds. So bleiben die Ausgabe und das einzelne Besitzstück getrennt.

Historische Angebote

Leihfrist, Versand, Abholadresse und Angebotsstatus stehen in einer eigenen Angebotsversion. Bei einer Änderung entsteht eine neue Zeile. Alte Ausleihen verweisen weiter auf die damals vereinbarte Version. Die bisherige Adresse bleibt erhalten. Ihr Inhalt darf nach der Verwendung in einer Version nicht mehr geändert werden.

Reservieren und ausleihen

Jede neue Ausleihe beginnt mit einer Anfrage. Erst die Bestätigung reserviert das Exemplar. Danach kann die Übergabe als Ausleihe erfasst werden. Auch eine sofortige Ausleihe nutzt diese Schritte. Anfrage, Bestätigung, Ablehnung, Stornierung und Umsetzung werden als Ereignisse festgehalten. Eine Anfrage allein blockiert das Buch noch nicht.

Verfügbarkeit und Rückgabe

Ein angebotenes Exemplar ist verfügbar, wenn weder eine laufende Ausleihe noch eine bestätigte Reservierung vorliegt. Eine überschrittene Leihfrist macht das Buch nicht automatisch frei. Entscheidend ist die eingetragene Rückgabe. Eine Bewertung ist erst danach erlaubt.

Regeln in der Datenbank

Primärschlüssel, Fremdschlüssel und Werteprüfungen sichern die Beziehungen. Trigger prüfen zusätzlich den Ablauf. Zwei partielle eindeutige Indizes verhindern mehrere offene Ausleihen oder Bestätigungen je Exemplar. Partielle Indizes gelten nur für Zeilen, die ihre Bedingung erfüllen (SQLite, 2022, Abschn. 2.1).

Die zyklische Beziehung zwischen Exemplar und aktueller Version wird erst beim Abschluss der Transaktion geprüft. Beide Zeilen können deshalb gemeinsam angelegt werden (SQLite, 2026, Abschn. 4.2).

Bestand und nachvollziehbarer Neuaufbau

Die aus den SQL-Dateien neu erstellte Datenbank enthält 16 Tabellen und 297 Datensätze. Jede Tabelle hat mindestens zehn Zeilen. Es gibt 24 Exemplare, 27 Angebotsversionen und 20 Reservierungen. Die zusätzlichen Statusereignisse entstehen durch die Trigger.

Tab. 1: Zeilen je Tabelle

Tabelle	Zeilen	Tabelle	Zeilen
abholzeit	48	adresse	12
angebot_version	27	ausleihe	14
autor	10	benutzer	12
bewertung	10	buch	12
buch_autor	13	buch_genre	13
exemplar	24	genre	10
reservierung	20	reservierungsereignis	52
sprache	10	verlag	10

Quelle: Eigene Zählung der neu erstellten Datenbank.

Die Datenbankdatei ist 221.184 Byte groß (216 KiB). Die logische Größe beträgt 54 Seiten mit je 4096 Byte. Der Messwert gilt für diesen Beispielbestand und SQLite 3.54.0. Er ist keine Größenprognose für einen echten Dienst.

Reproduzieren

Die ersten drei SQL-Dateien erstellen Tabellen, Regeln und Beispieldaten in einer Verbindung. Die weiteren Dateien enthalten Abfragen, Tests, Indexvergleiche und einen getrennten Altimport. Die Anleitung beschreibt die Reihenfolge und die nötigen SQLite-Funktionen. Frühere Phasen bleiben als eigener Arbeitsstand erhalten.

Die Dokumentation erklärt das Modell und die Regeln. Der vollständige SQL-Code liegt separat vor. Dadurch bleibt der Bericht kurz, während die Implementierung nachprüfbar ist.

Repository: https://github.com/siggismalz/iu-bookswap-datamart-sql

Prüfungen, Ergebnisse und Grenzen

Die 78 lokalen Datenbankprüfungen des bisherigen technischen Stands wurden am 14.10.2026 erneut ausgeführt. Alle bestanden; es gab keine Fehler. Die sechs neuen SQL-Ablaufprüfungen bestanden ebenfalls. Die Ergebnisse werden getrennt ausgewiesen; die Zahlen werden nicht addiert.

Die neuen Abläufe betreffen wiederholte Ausleihen, geänderte Bedingungen, Anfragen mit Stornierung, einen Versionswechsel nach Anfrage, das Aussetzen eines Angebots und Rollback. Weitere Prüfungen bestätigen erwartete Fehler, den Schutz bei zwei Verbindungen und den Altimport. Bei alten Ausleihen bleiben frühere Bedingungen ausdrücklich unbekannt.

Performance

Die bereits durchgeführte Messreihe vergleicht den Beispielbestand mit 1.000, 10.000 und 100.000 Ausleihen. Sie nutzt synthetische Daten und SQLite 3.45.3. Die folgenden Werte sind historische Messergebnisse. Der neue SQL-Vergleich ist separat ausführbar und liefert neue Werte für die jeweilige Umgebung.

Tab. 2: 100.000 Ausleihen: Median / P95 in ms

Abfrage	Alle Suchindizes	Ohne Suchindizes
Historie einer häufigen Person	90,0128 / 123,7238	79,6751 / 102,9636
Verfügbare Angebote	0,1164 / 0,1811	0,0511 / 0,0587
Reservierungen einer Person	0,1184 / 0,1986	6,3173 / 9,8576
Nähesuche, 5 km / 300 Treffer	10,9186 / 16,3433	5,1237 / 10,1034

Quelle: Eigene historische Messung; 30 Wiederholungen je Fall mit wiederverwendeter Verbindung.

Die Ergebnisse fallen unterschiedlich aus. Gezielte Filter nutzen alle 14 Suchindizes und benötigen dort weniger SQL-VM-Schritte. Das belegt nicht, dass jede Gesamtanfrage schneller wird. Bei der Nähesuche wird der Koordinatenindex nicht genutzt. Der verwendete Abfrageplan muss deshalb zusammen mit den Zeiten bewertet werden (SQLite, 2025, Abschn. 1.1).

Die Datenbank ersetzt keine fertige App. Anmeldung, Nachrichten, automatische Fristen, Prioritäten und reale Lasttests fehlen. Die Entfernung ist eine Luftlinie. Alte Verträge erhalten keine nachträglich erfundenen Bedingungen. Die synthetischen Tests zeigen, wie die Regeln in diesen Fällen funktionieren.

Quellen

SQLite. (2022, 18. April). *Partial indexes*. https://www.sqlite.org/partialindex.html

SQLite. (2025, 31. Mai). *EXPLAIN QUERY PLAN*. https://www.sqlite.org/eqp.html

SQLite. (2026, 20. März). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html